

FUNDAMENTOS DE TEORÍA DE LA COMPUTACIÓN 2026
Trabajo Práctico Nro 2

Comentario: Hacer mínimamente los ejercicios del 1 al 3. Los ejercicios 4 y 5 son un poco más difíciles; intentar hacerlos, porque se basan en una típica técnica relacionada con la clase RE.

Ejercicio 1. Responder breve y claramente los siguientes incisos:

1. ¿En qué se diferencian los lenguajes recursivos, los lenguajes recursivamente enumerables no recursivos y los lenguajes no recursivamente enumerables?

- Un lenguaje es **recursivo (R)** si y solo si existe una MT M que lo acepta y para siempre.
- Un lenguaje es **recursivamente enumerable (RE)** si y solo si existe una MT M que lo acepta. En este caso, no necesariamente va a parar (en casos negativos puede quedar loopeando).
- Un lenguaje es **no recursivamente enumerable (NRE)** si NO se puede construir una máquina de turing que lo reconozca ($\mathcal{L} - RE$ U CO-RE)

2. Probar que $R \subseteq RE \subseteq \mathcal{L}$. Ayuda: usar las definiciones.

Solo por definición, es trivial que $R \subseteq RE$.

Ya que la definición de RE es que exista una MT M que lo acepte, y la definición de R es que exista una MT M que lo acepte (lo mismo que RE) y que además pare siempre. Claramente R es un caso particular de RE, tiene una condición más. Pero SIEMPRE que está en R, está entonces también en RE.

Y $RE \subseteq \mathcal{L}$ porque RE es “el conjunto de lenguajes que tienen una MT que los acepta”, y $\mathcal{L}$ es el conjunto de todos los lenguajes.

Como RE contiene lenguajes, indefectiblemente RE está en $\mathcal{L}$.

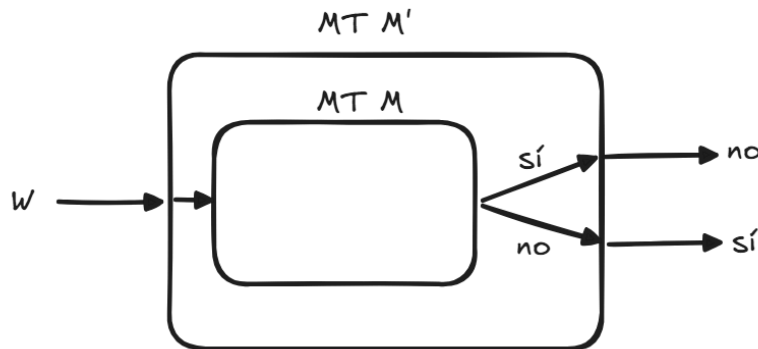
3. Dijimos en clase que el hecho de que si L es recursivo entonces L complemento también lo es, significa en términos de problemas que si un problema es decidible entonces también lo es el problema contrario. ¿Qué significa en términos de problemas que la intersección de dos lenguajes recursivos es también un lenguaje recursivo?

En términos de problemas, que la intersección de dos lenguajes recursivos sea también un lenguaje recursivo significa que la conjunción ($\vee$) de dos condiciones decidibles es, a su vez, decidible. (*conjunción de dos condiciones = un AND*)

Esto implica que si tenemos dos problemas independientes, P_1 y P_2 , para los cuales existe un algoritmo que siempre devuelve una respuesta (Sí/No) en tiempo finito, entonces el problema de determinar si una instancia cumple simultáneamente con ambas condiciones también tiene una solución algorítmica garantizada.

4. Explicar por qué no es correcta la siguiente prueba de que si $L \in RE$, también $L_c \in RE$: dada una MT M que acepta L , entonces la MT M' , igual que M pero con los estados finales permutados, **acepta** L_c .

Lo que plantea el enunciado es, graficamente, lo siguiente:



¿El problema? El enunciado nos dice que $L \in RE$.

Como ya vimos en el ejercicio 1.1, una máquina que reconoce un lenguaje de RE puede NO PARAR en casos negativos y quedarse loopeando. El esquema que grafiqué no consideró el escenario del loop, y ahí es donde está el error.

Si a MT M le damos algo de L_c y se queda loopeando (o sea, rechaza), MT M' también se queda loopeando (o sea, también rechaza, en vez de **aceptar** como asume la prueba incorrecta del enunciado).

(Además, si esta prueba fuera cierta, entonces R sería igual a RE (porque todo lenguaje RE tendría un complemento RE , y por teorema, si un lenguaje y su complemento son RE , entonces el lenguaje es R). Como sabemos que hay lenguajes en $RE - R$, ergo la prueba tiene que ser falsa.)

5. ¿Qué lenguajes de la clase CO-RE tienen MT que los aceptan? ¿También los deciden?

Solo los lenguajes de CO-RE que también pertenecen a RE (es decir, los lenguajes Recursivos R) tienen una MT que los acepte.

Y sí, al ser R, también los deciden.

(Que los decida quiere decir que siempre para)

Si un problema es **puramente CO-RE**, es **no reconocible**. No hay forma de construir una MT que te diga "Sí" cuando la cadena pertenece al lenguaje.

Lo único que podés hacer es construir una MT que reconozca cuando la cadena **no** pertenece (porque su complemento sí es RE).

En el momento en que inventás una MT que acepte un lenguaje CO-RE, no queda otra que sea **Recursivo (R)**, lo que significa que también es decidable.

6. Probar que el lenguaje Σ^* de todas las cadenas y el lenguaje vacío $\emptyset$ son recursivos. Alcanza con plantear la idea general. Ayuda: encontrar MT que los decidan.

Lenguaje Σ (todas las cadenas):

Σ^* contiene absolutamente todas las combinaciones posibles de símbolos del alfabeto universal

Idea General de la MT (Σ^*):

1. La máquina recibe una cadena w en la cinta.
2. Sin importar qué símbolos lea (o si la cinta está vacía), la máquina transiciona inmediatamente al estado de aceptación.

Como pudimos construir una máquina siempre se detiene y acepta, podemos asegurar que Σ^* es recursivo.

Lenguaje $\emptyset$:

Este lenguaje no contiene ninguna cadena (ni siquiera la cadena vacía λ). Es un conjunto sin elementos.

Idea General de la MT ($\emptyset$):

1. La máquina recibe una cadena w en la cinta.
2. Sin importar qué símbolos lea (o si la cinta está vacía), la máquina transiciona inmediatamente al **estado de rechazo**, no existe ningún w posible que pertenezca a $\emptyset$.

Como pudimos construir una máquina siempre se detiene y rechaza, podemos asegurar que $\emptyset$ es recursivo.

7. Probar que todo lenguaje finito es recursivo. Alcanza con plantear la idea general. Ayuda: encontrar una MT que lo decida (pensar cómo definir sus transiciones para cada una de las cadenas del lenguaje).

La idea es basicamente la de [este](#) proyecto de github jajaja, pero nosotros sí tenemos un conjunto finito.

Como nos dicen que el lenguaje es FINITO, hay una cantidad limitada de **combinaciones** (cadenas) que podemos formar con sus simbolos.

Ergo, podemos hacer una MT que “hardcodee” el lenguaje y listo. Compara una por una secuencialmente y si no es ninguna, rechaza.

Supongamos que el lenguaje finito es $L = \{w_1, w_2, w_3, \dots, w_n\}$.

Construimos una Máquina de Turing que funciona de la siguiente manera:

1. ¿La entrada es igual a w_1 ?
 - **SÍ:** ACEPTA.
 - **NO:** Pasa al siguiente estado.
2. ¿La entrada es igual a w_2 ?
 - **SÍ:** ACEPTA.
 - **NO:** Pasa al siguiente estado.
3. (...) Repetir para cada cadena hasta llegar a w_n .
4. ¿La entrada es igual a w_n ?
 - **SÍ:** ACEPTA.
 - **NO:** Si llegó hasta acá y no hubo coincidencia, **RECHAZA**.

Como es totalmente posible construir una máquina que siempre, eventualmente, se detiene y decide, podemos asegurar que todo lenguaje finito es recursivo.

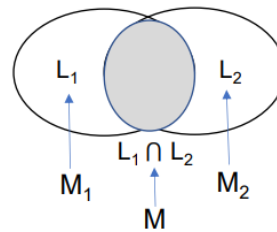
Ejercicio 2. Considerando la Propiedad 2 estudiada en clase:

Propiedad 2. Si $L_1 \in R$ y $L_2 \in R$, entonces $L_1 \cap L_2 \in R$.

Es decir, si existe una MT M_1 que decide L_1 ,

y existe una MT M_2 que decide L_2 ,

también existe una MT M que decide $L_1 \cap L_2$.



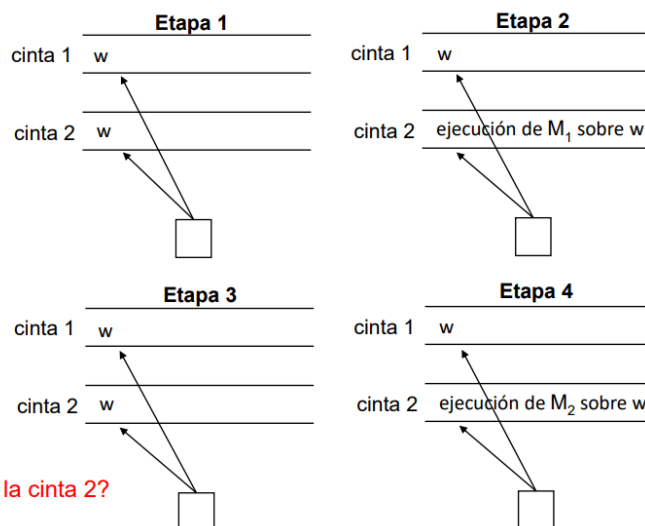
Construir una MT M que ejecute secuencialmente las MT M_1 y M_2 y acepte sii M_1 y M_2 aceptan.

Idea de la construcción de la MT M .

M tiene 2 cintas. Con w en la cinta 1, hace:

1. Copia w en la cinta 2.
2. Ejecuta M_1 sobre w en la cinta 2.
Si M_1 para en q_R ,
entonces M para en q_R .
3. Borra el contenido de la cinta 2
y copia de nuevo w en la cinta 2.
4. Ejecuta M_2 sobre w en la cinta 2.
Si M_2 para en $q_A(q_R)$,
entonces M para en $q_A(q_R)$.

Ejercicio: ¿cómo hace M para copiar w y borrar la cinta 2?



1. ¿Cómo implementaría la copia de la entrada w en la cinta 2 de la MT M ?

1. Leo caracter de la cinta 1, lo escribo en la cinta 2. Muevo a derecha en ambas cintas.
2. Repito paso 1 mientras el caracter leído sea distinto a B (blanco).
3. Vuelvo moviendome hacia la izquierda en ambas cintas hasta encontrar un B en la cinta 1. Y doy un solo pasito a la derecha, para quedar correctamente posicionado.

2. ¿Cómo implementaría el borrado del contenido de la cinta 2 de la MT M?

En la ejecución de M1 sobre la cinta 2 pueden escribirse blancos y el cabezal terminar en cualquier lado, dificultando saber hasta donde y hacia qué lado borrar.

Entonces para implementarlo hay que "encerrar" el espacio de trabajo de M1 entre dos símbolos especiales que no pertenezcan al alfabeto de la cinta (Σ), por ejemplo, $\vdash$ para el inicio y $\dashv$ para el fin.

Idea General:

1. Al copiar w de la Cinta 1 a la Cinta 2, se escribe $\vdash$ al principio y $\dashv$ inmediatamente después del último carácter de w .
2. La MT M ejecuta M1. Si en algún momento M_1 intenta escribir sobre alguna marca, la MT M debe desplazar la marca a la izquierda o a la derecha.
3. **Para borrar:**
 - El cabezal de la Cinta 2 busca la marca inicial $\vdash$
 - Escribe B para borrar la marca y se desplaza hacia la derecha.
 - Repite el paso anterior escribiendo B (en todos los símbolos de w) hasta encontrar $\dashv$
 - Al encontrar la marca, se escribe un B sobre ella y termina el borrado.

Ejercicio 3. Probar:

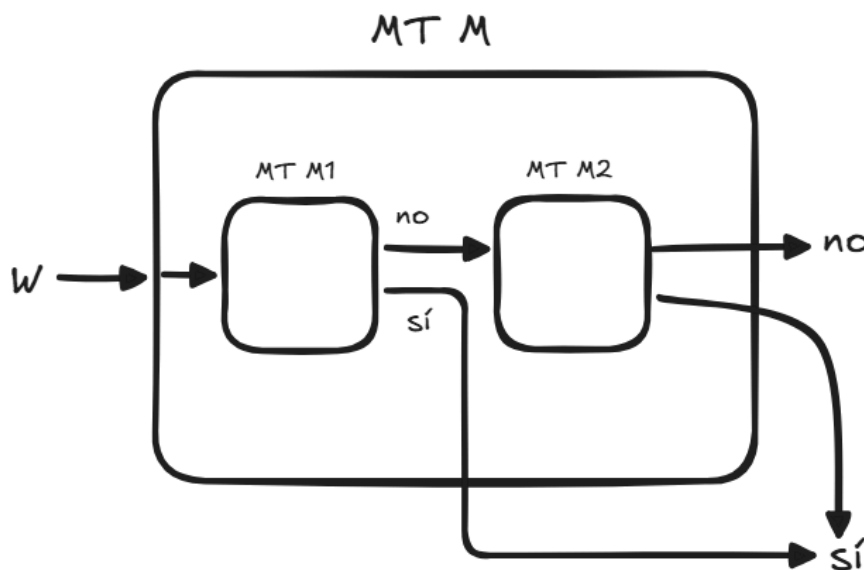
1. La clase R es cerrada con respecto a la operación de unión. Ayuda: la prueba es similar a la desarrollada para la intersección.

Dadas dos máquinas MT M1 y MT M2, que aceptan L1 y L2 y paran siempre, se puede construir una máquina M que acepte $L1 \cup L2$ y pare siempre.

Idea general:

M tiene 2 cintas. Con w en la cinta 1, hace:

1. Copia w en la cinta 2.
2. Ejecuta M1 sobre w en la cinta 2.
Si M1 para en q_A , entonces M para en q_A .
Si no, sigue con el paso 3.
3. Borra el contenido de la cinta 2 y copia de nuevo w en la cinta 2.
4. Ejecuta M2 sobre w en la cinta 2.
Si M2 para en q_A (o q_R), entonces M para en q_A (o q_R).



2. La clase RE es cerrada con respecto a la operación de intersección.

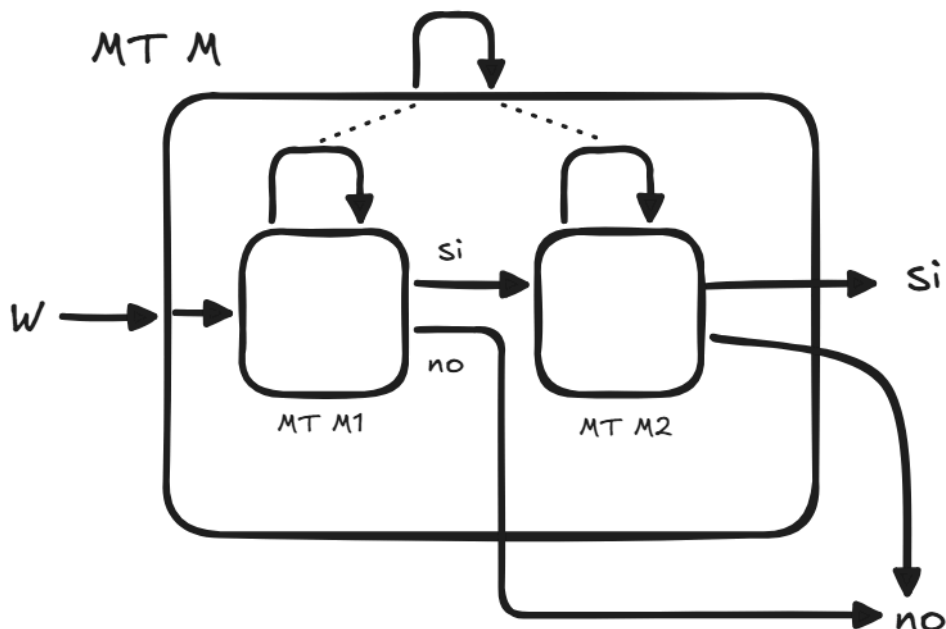
Ayuda: la prueba es similar a la desarrollada para la clase R.

Dadas dos máquinas MT M1 y MT M2, que aceptan L1 y L2, se puede construir una máquina M que acepte $L1 \cap L2$.

Idea general:

M tiene 2 cintas. Con w en la cinta 1, hace:

1. Copia w en la cinta 2.
2. Ejecuta M1 sobre w en la cinta 2.
Si M1 para en q_R , entonces M para en q_R .
Si M1 loopea, entonces M quedará loopeando.
Si M1 para en q_A , entonces M sigue con el paso 3.
3. Borra el contenido de la cinta 2 y copia de nuevo w en la cinta 2.
4. Ejecuta M2 sobre w en la cinta 2.
Si M2 para en q_A (o q_R), entonces M para en q_A (o q_R).
Si M2 loopea, entonces M quedará loopeando.



Ejercicio 4. Sean L_1 y L_2 dos lenguajes recursivamente enumerables de números naturales codificados en unario (por ejemplo, el número 5 se representa con 11111). Probar que también es recursivamente enumerable el lenguaje $L = \{x \mid x \text{ es un número natural codificado en unario, y existen } y, z, \text{ tales que } y + z = x, \text{ con } y \in L_1, z \in L_2\}$. Ayuda: la prueba es similar a la vista en clase, de la clausura de la clase RE con respecto a la operación de concatenación.

Lo unico difícil de este ejercicio es entender el enunciado.

Está pidiendo que sumemos dos numeros naturales codificados en unario.

Para sumar dos numeros unarios basta con concatenarlos, cosa que ya está resuelta en la explicación práctica. Ni hay que pensar, es copiar.

El problema:

Como no sabemos cuales son esos 2 numeros (y y z) hay que particionar, ir moviendo el centro, y probar una por una.

Ejemplo con strings: "hola" y "chau"

prueba 1: {"" + "holachau"}

prueba 2: {"h" + "olachau"}

prueba 3: {"ho" + "lachau"}

etc...

Si "ho" pertenece a L_1 , y "lachau" pertenece a L_2 , da qA en ese 3er chequeo.

Si al final no hubo coincidencia, da qR.

Eso mismo pero con unario, para ver si encontramos los numeros y y z que sumados dan nuestro input x

Entonces, procedo...

Idea general:

Se tiene que construir una MT M que acepte L ejecutando sobre un input x (un número natural de n símbolos '1') determinadas MT M_1 y M_2 (que aceptan L_1 y L_2 respectivamente, las cuales pueden looppear en casos negativos).

Como estamos en la clase RE, **no podemos probar una partición tras otra de forma secuencial**, ya que una partición incorrecta podría hacer que M_1 o M_2 entren en un bucle infinito, impidiendo que probemos la partición correcta.

La idea sigue siendo probar particiones, pero hay que hacerlo de forma paralela.

- M primero debe hacer ejecuciones de 1 paso de M_1 y M_2 con todas las posibles particiones (y, z) tales que $y+z=x$.
 - Luego, ejecuciones de 2 pasos con todas las particiones.
 - Luego, ejecuciones de 3 pasos con todas las particiones, y así siguiendo.
- Si para alguna partición específica (y_i, z_i) , tanto M_1 como M_2 llegan a sus estados de aceptación, entonces M acepta eventualmente.

Es como un round robin de ISO

Ejercicio 5. Dada una MT M1 con alfabeto $\Gamma = \{0, 1, B\}$:

1. Construir una MT M2, utilizando la MT M1, que acepte, cualquiera sea su cadena de entrada, sii la MT M1 acepta **al menos** una cadena.

Ayuda para la parte (1): si M1 acepta al menos una cadena, entonces existe al menos una cadena de símbolos 0 y 1, de tamaño n, tal que M1 la acepta en k pasos. Teniendo en cuenta esto, pensar cómo M2 podría simular M1 considerando todas las cadenas de símbolos 0 y 1 hasta encontrar eventualmente una que M1 acepte (¡cuidándose de los casos en que M1 entre en loop!).

Similar a lo de concatenación del ejercicio anterior, pero con otra vuelta de tuerca. En el ejercicio 4 teníamos que paralelizar dándole N pasos a cada partición, porque estábamos en RE.

Ahora no sabemos cómo está construida M1, así que podría aceptar 0, 1 o muchas cadenas, quiero decir, podría looppear. Claramente también hay que paralelizar. Pero cuál es el quilombo extra? En el ejercicio 4 teníamos opciones finitas a probar (dada por la longitud del input N, en mi ejemplo “holachau” era un string finito de 8 caracteres). Ahora en este nuevo ejercicio tenemos INFINITAS posibilidades de números binarios a chequear.

Ergo, para que M2 funcione, tiene que intercalar no solo los pasos, sino también las cadenas.

1: Ejecuta un paso de M1 con la cadena 1.

2: Ejecuta dos pasos de M1 con la cadena 1, y un paso con la cadena 2.

3: Ejecuta tres pasos con la cadena 1, dos pasos con la cadena 2, y un paso con la cadena 3.

... y así sucesivamente, formando una diagonal.

Si en algún momento la simulación de M1 acepta **cualquier** cadena de las que está probando, entonces M2 se detiene y acepta, ya que el enunciado dice que probemos **al menos** una.

2. ¿Se puede construir además una MT M3, utilizando la MT M1, que acepte, cualquiera sea su cadena de entrada, sii la MT M1 acepta a lo sumo una cadena? Justificar.

Para que una máquina acepte que "a lo sumo hay una cadena", debería poder confirmar que no existe una segunda cadena.

- Si M1 acepta dos cadenas , eventualmente las encontraríamos (eso es un problema RE).
- Pero si M1 acepta solo una, nunca podríamos estar seguros de que no hay otra "escondida" que tardaría un trillón de años en ser aceptada. Tendríamos que esperar por siempre.

Si existen infinitas combinaciones de cadenas que M1 puede aceptar deberíamos de probar para una misma entrada w todas las posibles combinaciones para determinar que SOLO una de ellas la acepta (considerar que si bien podemos tener una cadena que la acepte deberíamos verificar que todas las demás las rechacen). El único caso que podría parar esta MT sería al rechazar, si se aceptan dos cadenas.

(esto se ve en la clase y practica 3, de indecibilidad)

Como es una propiedad no trivial de los lenguajes RE, por el **Teorema de Rice**, sabemos que es indecible. Este problema está en CO-RE
Similar al halting problem? creo (?)